TP 0 – Python Basics for Cryptography

Master 1 : Cybersecurity & Artificial Intelligence

Year: 2025/2026 KADDACHE M. E. A

الأهداف

- تعلم العمليات العنصرية الأساسية المباشرة في بايثون (Elementary Operation)
- تعلم الدوال المعرفة مسبقا المفيدة في التشفير
- تكوين علبة أدوات دائمة للحصص القادمة

To avoid repeatedly running the code, use the Python terminal by typing python commands in the CMD, or use repeated execution if that is not possible

Part 01 :
Base Conversion

String to number :

```
>>> int("1011", 2)     # ثنائي → عشري
>>> int("17", 8)       # ثماني
>>> int("1f", 16)      # سداسي عشري
```

Number to base :

```
>>> bin(10)   # '0b1010'
>>> oct(10)   # '0o12'
>>> hex(10)   # '0xa'
```

format() :

```
>>> format(10, 'b')   # '1010'
>>> format(10, 'o')   # '12'
>>> format(10, 'x')   # 'a'
```

Customize conversion :

```
>>> format(65, '08b')
>>> format(65, '02x')
>>> format(65, '04x')
>>> format(65, '09b')
>>> format(65, '016b')
```

Byte + format :

```
>>> byte = 0x41
>>> format(byte, '08b')
>>> format(byte, '08o')
>>> format(byte, '04d')
>>> format(byte, '08d')
```

Bytes + bytearray:

```
>>> b = bytes([65, 66, 67])
>>> b
```

From hex :

```
>>> b=bytes.fromhex("41 42 43")
>>> b
```

To hex :

```
>>> b.hex()
```

Bytearray (editable) :

```
>>> ba = bytearray(b"ABC")
>>> ba[0] = 0x44
>>> ba
```

Part 02 :
Elementary Operation

XOR:

```
>>> a=10
>>> b=20
>>> a^b  #a xor b decimal
```

- with code execution use ***print()*** like :

```
a=10
b=20
print(a^b)
```

Binary XOR :

```
>>> a=0b1010
>>> b=0b10100
>>> bin(a^b)
```

Left and Right Shift :

```
>>> 5 >> 1
>>> 5 << 1
>>> 10 << 10
>>> 10 >> 10
>>> 10 >> 2
```

- Use **bin()** to to see what is happening in the binary format

```
a= 10 * 2**10
b= 10 << 10
a == b
```

- **a == b** true or false and why ?
- What is the equivalent of the operation **10 >> 10**?
- What are bit shift operations useful for and why?

OR (|) – AND (&) – NOT (~) :

```
>>> a=301
>>> b=410
>>> a | b
>>> a & b
>>> ~a
>>> ~a | a
>>> ~a & a
>>> ~a & b
```

~ : Bitwise negation (2 complement) = -(a+1)

Part 03 :
ASCII – UTF8 - Base64

Characters & ASCII :

```
>>> ord('A')
>>> chr(65)
```

- **Simplify** $P(x) = ord(chr(x)) - chr(ord(x))$

Cryptography always works on **bytes**, not strings :

```
b = "HELLO".encode('utf-8')
s = b.decode('utf-8')
```

Base64 :

```
import base64

data = b"m1cysia"
encoded = base64.b64encode(data)
decoded = base64.b64decode(encoded)

print(encoded)
print(decoded)
```

- Change the content of data multiple times and identify the Base64 character set
- In how many bits is the Base64 character set represented in binary?

Base32 :

```
import base64
data = b"CyberSecurity"
print(base64.b32encode(data))
```

Base85:

```
import base64
data = b"CyberSecurity"
print(base64.b85encode(data))
```

Part 04 :
Pseudo-random and modular arithmetic

modular arithmetic :

```
>>> pow(1001001010,10019910,513) # pow(base,expo,modulo)
```

**** VS pow :**

$$2^n = 0 \ mod \ 2 \ (\ directly \ and \ without \ thinking \ )$$

So test this :

```
>>> (2**1000000)%2
>>> (2**10000000)%2
>>> (2**100000000)%2
>>> (2**1000000000)%2
```

Test this :

```
>>> pow(2,1000000000000000000000000000000,2)
>>> pow(2,1000000000000000000000000000000000000000000,2)
```

Random :

To create random number with N bit :

```
N=10 # 10 bit
r=secrets.randbits(N) # r random number of 10 bit
```

test code

```
import secrets
i=1
while i<256 :
    x = secrets.randbits(i)
    print(x)
    i=i*2
```

2/

```
import secrets

x = secrets.randbits(2048)
print(x)
```

Exercices

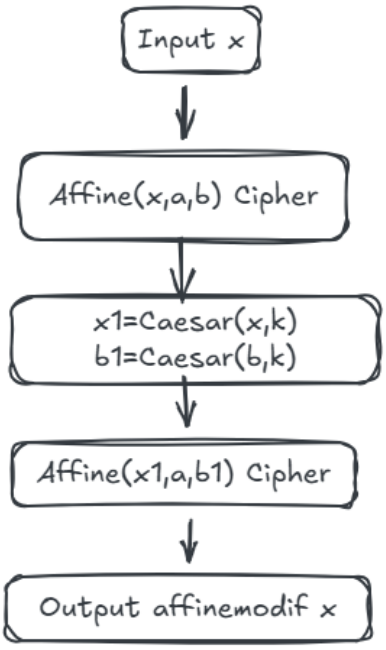
Exo 1:

Write a Python program to create a Caesar encryption

```python
def Caesar(text : str ,key : int):
    cipher=''
    #your code
    return cipher
```

Exo 2 :

Write a Python program to create a `AffineModif(t:str,a:int,b:int,k:int)` encryption




```python
def Caesar(text : str ,key : int):
    cipher=''
    #your code
    return cipher

def affine(t : str ,a : int , b : int):
    cipher=''
    #your code
    return cipher

def AffineModif( t : str ,a,b,k : int):
    cipher=''
    #your code
    return cipher
```

- Make **affinemodif** work on the full ASCII range (modulo 256), not just 26.
- Write a decryption function **affinemodif-1**.
- Analyze the **strength of the cipher** and determine how many **brute-force attempts** are required to break it

---

Welcome To Semantic Security

KADDACHE Mohammed El Amine